

Российский университет дружбы народов

Факультет физико-математических и естественных наук

Кафедра компьютерных и информационных наук

ОТЧЕТ

ПО ЛАБОРАТОРНОЙ РАБОТЕ №7

Дисциплина: Архитектура компьютеров

Студент: Алексеев Тимофей

Группа: НКАбд-07-25

Москва

2025 г.

Содержание

1. Цель работы	5
2. Задание	6
3. Теоретическое введение	7
4. Выполнение лабораторной работы	8
4.1 Реализация переходов в NASM	8
4.2 Изучение структуры файла листинга	15
4.3 Задания для самостоятельной работы	18
5. Выводы	24
6. Список литературы	25

Список иллюстраций

4.1 Создание каталога и файла для программы	8
4.2 Сохранение программы	9
4.3 Запуск программы	9
4.4 Изменение программы	10
4.5 Запуск измененной программы	11
4.6 Изменение программы	12
4.7 Проверка изменений	13
4.8 Сохранение новой программы	14
4.9 Проверка программы из листинга	15
4.10 Проверка файла листинга	16
4.11 Удаление операнда из программы	17
4.12 Просмотр ошибки в файле листинга	18
4.13 Первая программа самостоятельной работы	18
4.14 Проверка работы первой программы	20
4.15 Вторая программа самостоятельной работы	21
4.16 Проверка работы второй программы	23

Список таблиц

1. Цель работы

Изучение команд условного и безусловного переходов. Приобретение навыков написания программ с использованием переходов. Знакомство с назначением и структурой файла листинга.

2. Задание

1. Реализация переходов в NASM
2. Изучение структуры файлов листинга
3. Самостоятельное написание программ по материалам лабораторной работы

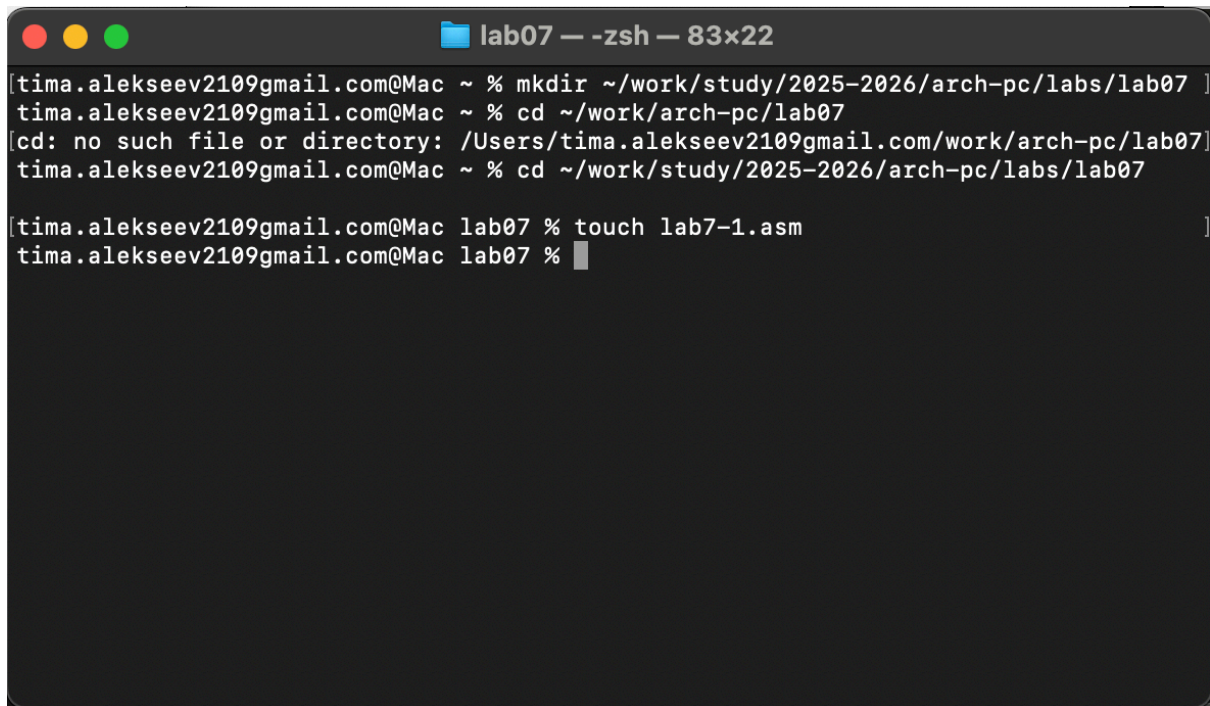
3. Теоретическое введение

Для реализации ветвлений в ассемблере используются так называемые команды передачи управления или команды перехода. Можно выделить 2 типа переходов: условный переход – выполнение или не выполнение перехода в определенную точку программы в зависимости от проверки условия. Безусловный переход – выполнение передачи управления в определенную точку программы без каких-либо условий.

4. Выполнение лабораторной работы

4.1 Реализация переходов в NASM

Создаю каталог для программ лабораторной работы №7 (рис. 4.1).

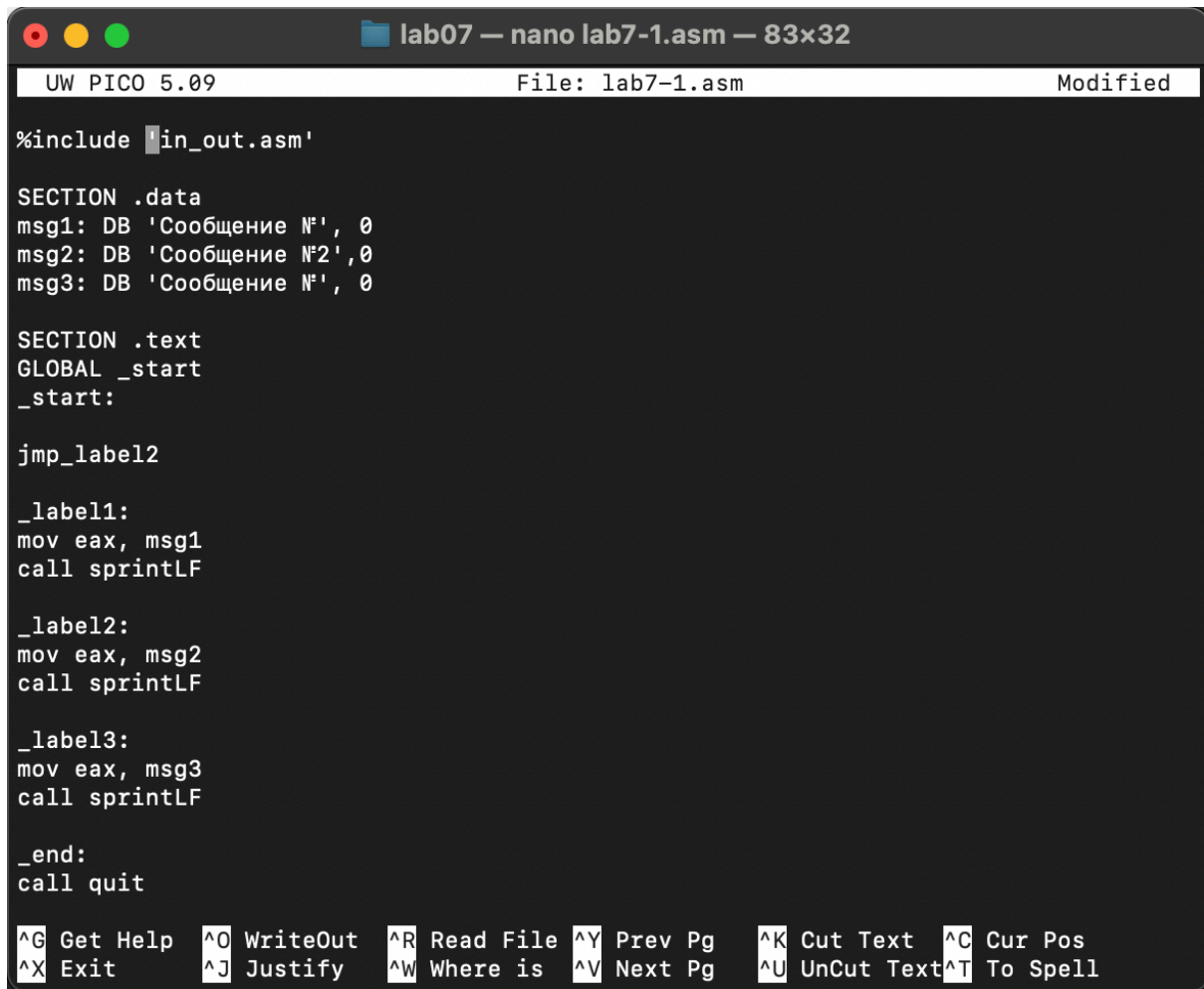


```
lab07 — -zsh — 83x22
[tima.alekseev2109gmail.com@Mac ~ % mkdir ~/work/study/2025-2026/arch-pc/labs/lab07 ]
tima.alekseev2109gmail.com@Mac ~ % cd ~/work/arch-pc/lab07
[cd: no such file or directory: /Users/tima.alekseev2109gmail.com/work/arch-pc/lab07]
tima.alekseev2109gmail.com@Mac ~ % cd ~/work/study/2025-2026/arch-pc/labs/lab07

[tima.alekseev2109gmail.com@Mac lab07 % touch lab7-1.asm ]
tima.alekseev2109gmail.com@Mac lab07 %
```

Рис. 4.1: Создание каталога и файла для программы

Копирую код из листинга в файл будущей программы. (рис. 4.2).



The screenshot shows a nano editor window titled "lab07 — nano lab7-1.asm — 83x32". The status bar at the top indicates "UW PICO 5.09", "File: lab7-1.asm", and "Modified". The code content is as follows:

```
%include 'in_out.asm'

SECTION .data
msg1: DB 'Сообщение №1', 0
msg2: DB 'Сообщение №2', 0
msg3: DB 'Сообщение №3', 0

SECTION .text
GLOBAL _start
_start:

jmp_label2

_label1:
mov eax, msg1
call sprintLF

_label2:
mov eax, msg2
call sprintLF

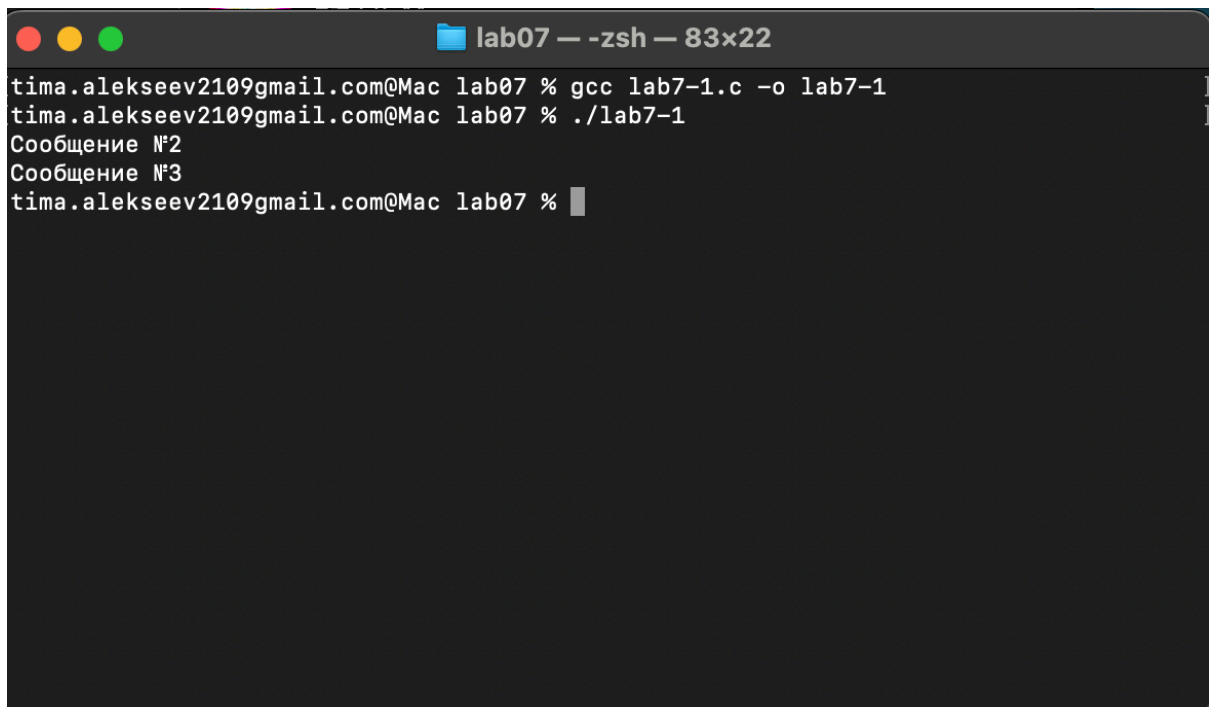
_label3:
mov eax, msg3
call sprintLF

_end:
call quit
```

At the bottom, a row of keyboard shortcuts is displayed: **^G** Get Help, **^O** WriteOut, **^R** Read File, **^Y** Prev Pg, **^K** Cut Text, **^C** Cur Pos, **^X** Exit, **^J** Justify, **^W** Where is, **^V** Next Pg, **^U** UnCut Text, **^T** To Spell.

Рис. 4.2: Сохранение программы

При запуске программы я убедился в том, что безусловный переход действительно изменяет порядок выполнения инструкций (рис. 4.3).



The screenshot shows a terminal window titled "lab07 — -zsh — 83x22". The prompt is "tima.alekseev2109gmail.com@Mac lab07 %". The user has entered the following commands:

```
gcc lab7-1.c -o lab7-1
./lab7-1
```

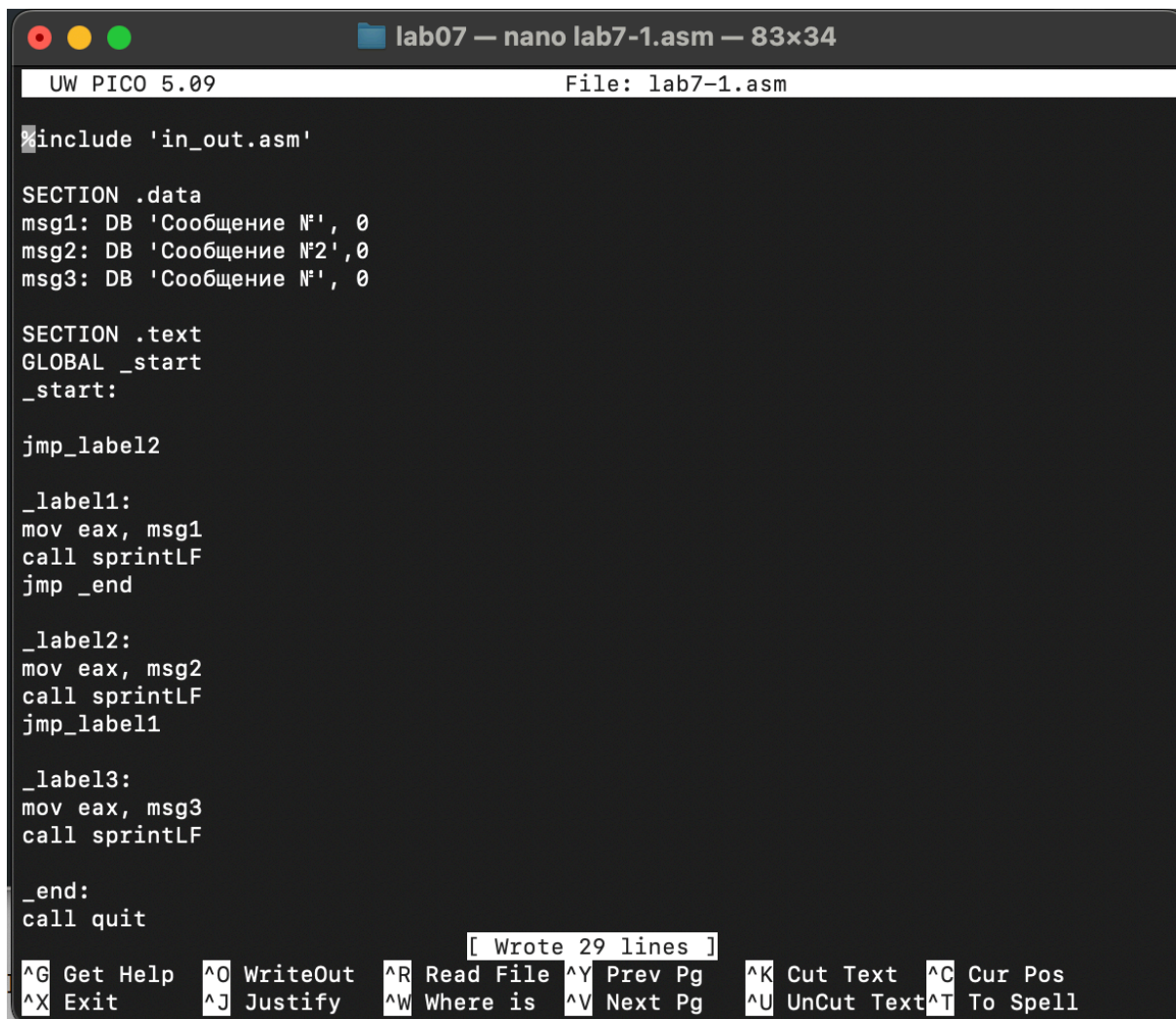
The output of the program is:

```
Сообщение №2
Сообщение №3
```

The prompt is now "tima.alekseev2109gmail.com@Mac lab07 %".

Рис. 4.3: Запуск программы

Изменяю программу таким образом, чтобы поменялся порядок выполнения 9 функций (рис. 4.4).



```
lab07 — nano lab7-1.asm — 83x34
UW PICO 5.09 File: lab7-1.asm

#include 'in_out.asm'

SECTION .data
msg1: DB 'Сообщение №1', 0
msg2: DB 'Сообщение №2', 0
msg3: DB 'Сообщение №1', 0

SECTION .text
GLOBAL _start
_start:

jmp_label2

_label1:
mov eax, msg1
call sprintLF
jmp _end

_label2:
mov eax, msg2
call sprintLF
jmp_label1

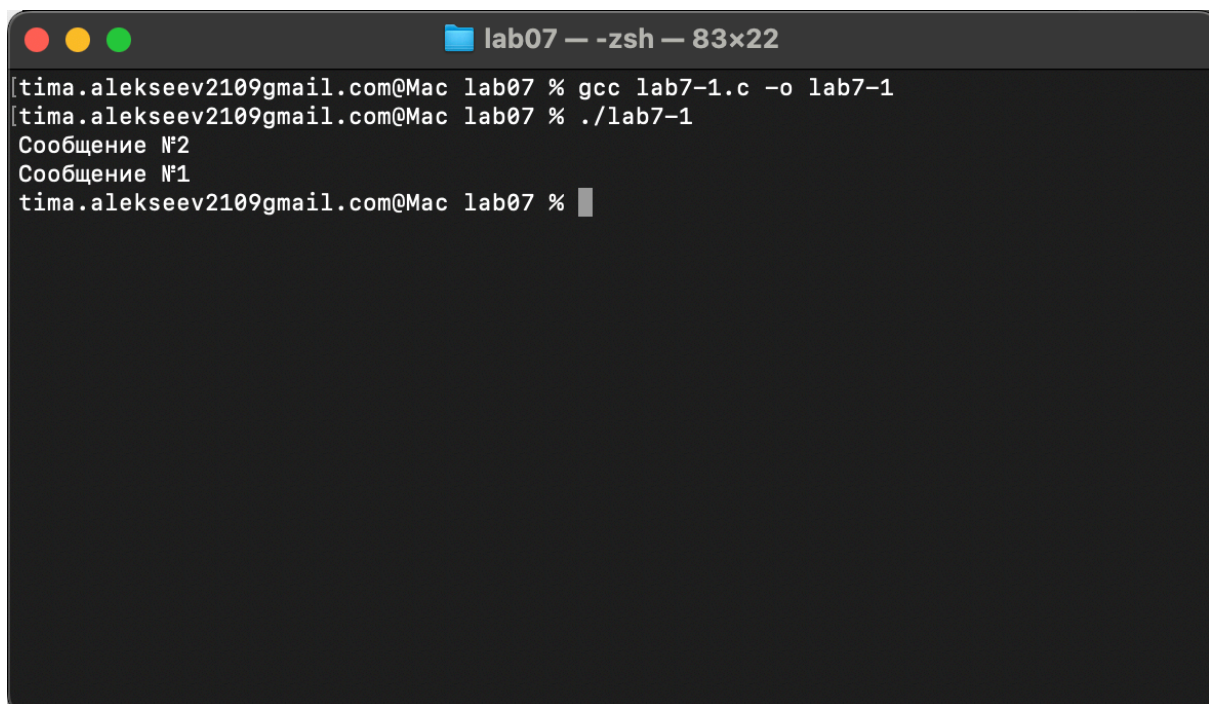
_label3:
mov eax, msg3
call sprintLF

_end:
call quit

[ Wrote 29 lines ]
^G Get Help ^O WriteOut ^R Read File ^Y Prev Pg ^K Cut Text ^C Cur Pos
^X Exit ^J Justify ^W Where is ^V Next Pg ^U UnCut Text ^T To Spell
```

Рис. 4.4: Изменение программы

Запускаю программу и проверяю, что примененные изменения верны (рис. 4.5).

A terminal window titled "lab07 — -zsh — 83x22" with standard macOS window controls (red, yellow, green buttons). The terminal shows the following commands and output:

```
[tima.alekseev2109gmail.com@Mac lab07 % gcc lab7-1.c -o lab7-1  
[tima.alekseev2109gmail.com@Mac lab07 % ./lab7-1  
Сообщение №2  
Сообщение №1  
tima.alekseev2109gmail.com@Mac lab07 %
```

Рис. 4.5: Запуск измененной программы

Теперь изменяю текст программы так, чтобы все три сообщения вывелись в обратном порядке (рис. 4.6).



```
lab07 — nano lab7-1.asm — 83x35
UW PICO 5.09                               File: lab7-1.asm

#include 'in_out.asm'

SECTION .data
msg1: DB 'Сообщение №', 0
msg2: DB 'Сообщение №2', 0
msg3: DB 'Сообщение №', 0

SECTION .text
GLOBAL _start
_start:

jmp_label3

_label1:
mov eax, msg1
call sprintLF
jmp _end

_label2:
mov eax, msg2
call sprintLF
jmp_label1

_label3:
mov eax, msg3
call sprintLF
jmp _label2

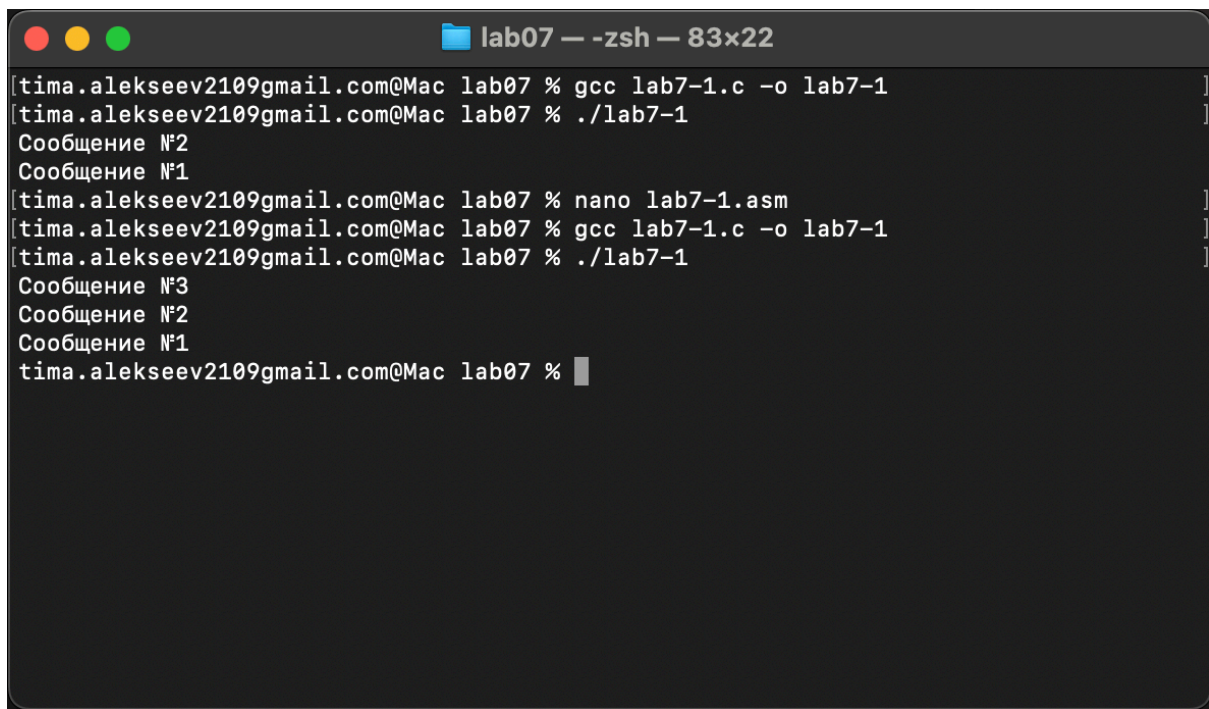
_end:
call quit
```

[Wrote 30 lines]

^G Get Help	^O WriteOut	^R Read File	^Y Prev Pg	^K Cut Text	^C Cur Pos
^X Exit	^J Justify	^W Where is	^V Next Pg	^U UnCut Text	^T To Spell

Рис. 4.6: Изменение программы

Работа выполнена корректно, программа в нужном мне порядке выводит сообщения (рис. 4.7).

A terminal window titled "lab07 — -zsh — 83x22" with standard macOS window controls (red, yellow, green buttons). The terminal shows a series of commands and their outputs. The user runs 'gcc lab7-1.c -o lab7-1', then './lab7-1'. The program outputs "Сообщение №2" followed by "Сообщение №1". The user then runs 'nano lab7-1.asm', followed by 'gcc lab7-1.c -o lab7-1' and './lab7-1'. The program outputs "Сообщение №3", "Сообщение №2", and "Сообщение №1". The terminal ends with a prompt "tima.alekseev2109gmail.com@Mac lab07 %" and a cursor.

```
[tima.alekseev2109gmail.com@Mac lab07 % gcc lab7-1.c -o lab7-1 ]
[tima.alekseev2109gmail.com@Mac lab07 % ./lab7-1 ]
Сообщение №2
Сообщение №1
[tima.alekseev2109gmail.com@Mac lab07 % nano lab7-1.asm ]
[tima.alekseev2109gmail.com@Mac lab07 % gcc lab7-1.c -o lab7-1 ]
[tima.alekseev2109gmail.com@Mac lab07 % ./lab7-1 ]
Сообщение №3
Сообщение №2
Сообщение №1
tima.alekseev2109gmail.com@Mac lab07 % █
```

Рис. 4.7: Проверка изменений

Создаю новый рабочий файл и вставляю в него код из следующего листинга (рис. 4.8).


```
lab07 — nano lab7-2.asm — 83x56
UW PICO 5.09 File: lab7-2.asm

#include 'in_out.asm'

SECTION .data
msg1 db 'Введите B: ', 0h
msg2 db 'Наибольшее число: ', 0h
A dd '20'
C dd '50'

SECTION .bss
max resb 10
B resb 10

SECTION .text
GLOBAL _start
_start:
    mov eax, msg1
    call sprint

    mov ecx, B
    mov edx, 10
    call sread

    mov eax, B
    call atoi
    mov [B], eax

    mov ecx, [A]
    mov [max], ecx

    cmp ecx, [C]
    jg check_B
    mov ecx, [C]
    mov [max], ecx

check_B:
    mov eax, max
    call atoi
    mov [max], eax

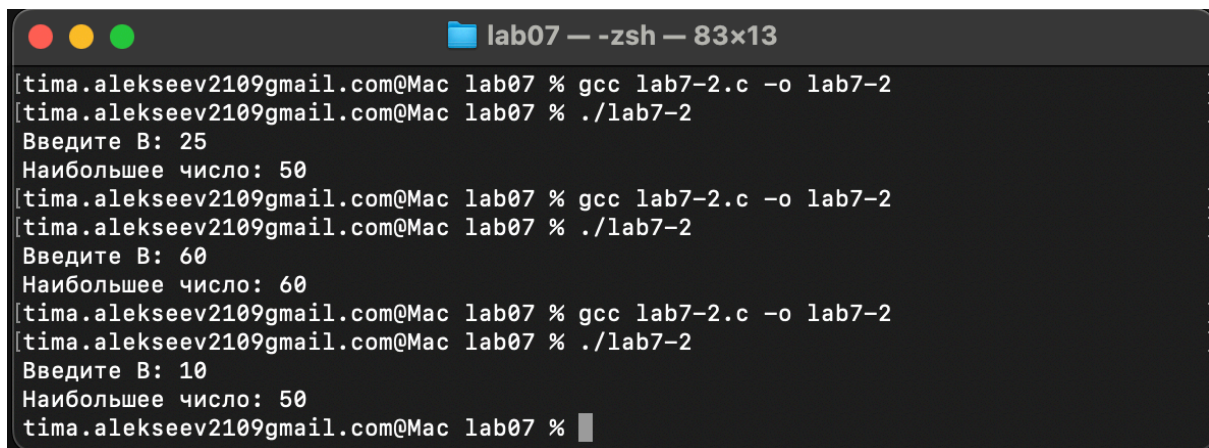
    mov ecx, [max]
    cmp ecx, [B]
    jg fin
    mov ecx, [B]
    mov [max], ecx

fin:
    mov eax, msg2
    call sprint
    mov eax, [max]
    call iprintLF
    call quit

[ Wrote 52 lines ]
^G Get Help ^O WriteOut ^R Read File ^Y Prev Pg ^K Cut Text ^C Cur Pos
^X Exit ^J Justify ^W Where is ^V Next Pg ^U UnCut Text ^T To Spell
```

Рис. 4.8: Сохранение новой программы

Программа выводит значение переменной с максимальным значением, проверяю работу программы с разными входными данными (рис. 4.9).

A terminal window titled 'lab07 — -zsh — 83x13' showing the execution of a program. The user runs 'gcc lab7-2.c -o lab7-2' and then './lab7-2'. The program prompts for input 'Введите B:' and outputs 'Наибольшее число: 50'. This process is repeated three times with inputs 25, 60, and 10, all resulting in an output of 50.

```
lab07 % gcc lab7-2.c -o lab7-2
lab07 % ./lab7-2
Введите B: 25
Наибольшее число: 50
lab07 % gcc lab7-2.c -o lab7-2
lab07 % ./lab7-2
Введите B: 60
Наибольшее число: 60
lab07 % gcc lab7-2.c -o lab7-2
lab07 % ./lab7-2
Введите B: 10
Наибольшее число: 50
lab07 %
```

Рис. 4.9: Проверка программы из листинга

4.2 Изучение структуры файла листинга

Создаю файл листинга с помощью флага -l команды `pasn` и открываю его с помощью текстового редактора `mousepad` (рис. 4.10).

```
%include 'in_out.asm'

;----- slen -----
; Функция вычисления длины сообщения
slen:
    push    ebx
    mov     ebx, eax
nextchar:
    cmp     byte [eax], 0
    jz      finished
    inc     eax
    jmp     nextchar
finished:
    sub     eax, ebx
    pop     ebx
    ret

;----- sprint -----
; Функция печати сообщения
; входные данные: mov eax,<message>
sprint:
    push    edx
    push    ecx
    push    ebx
    push    eax
    call    slen
    mov     edx, eax
    pop     eax
    mov     ecx, eax
    mov     ebx, 1
    mov     eax, 4
    int     80h
    pop     ebx
    pop     ecx
    pop     edx
    ret

;----- sprintf -----
; Функция печати сообщения с переводом строки
; входные данные: mov eax,<message>
sprintf:
    call    sprint
    push    
```

Рис. 4.10: Проверка файла листинга

Первое значение в файле листинга - номер строки, и он может вовсе не совпадать с номером строки изначального файла. Второе вхождение - адрес, смещение машинного кода относительно начала текущего сегмента, затем непосредственно идет сам машинный код, а заключает строку исходный текст программы с комментариями. Удаляю один операнд из случайной инструкции, чтобы проверить поведение файла листинга в дальнейшем (рис. 4.11).


```

lab07 — nano lab7-2.asm — 83x47
UW PICO 5.09 File: lab7-2.asm

resb 10

SECTION .text
GLOBAL _start
_start:

    mov eax, msg1
    call sprint

    mov ecx, B
    mov edx, 10
    call sread

    mov eax, B
    call atoi
    mov [B], eax

    mov ecx, [A]
    mov [max], ecx

    cmp ecx, [C]
    jg check_B
    mov ecx, [C]
    mov [max], ecx

check_B:
    mov eax, |
    call atoi
    mov [max], eax

    mov ecx, [max]
    cmp ecx, [B]
    jg fin
    mov ecx, [B]
    mov [max], ecx

fin:
    mov eax, msg2
    call sprint
    mov eax, [max]
    call iprintLF
    call quit

[ Wrote 43 lines ]
^G Get Help ^O WriteOut ^R Read File ^Y Prev Pg ^K Cut Text ^C Cur Pos
^X Exit ^J Justify ^W Where is ^V Next Pg ^U UnCut Text ^T To Spell

```

Рис. 4.11: Удаление операнда из программы

В новом файле листинга показывает ошибку, которая возникла при попытке трансляции файла. Никакие выходные файлы при этом помимо файла листинга не создаются. (рис. 4.12).

```

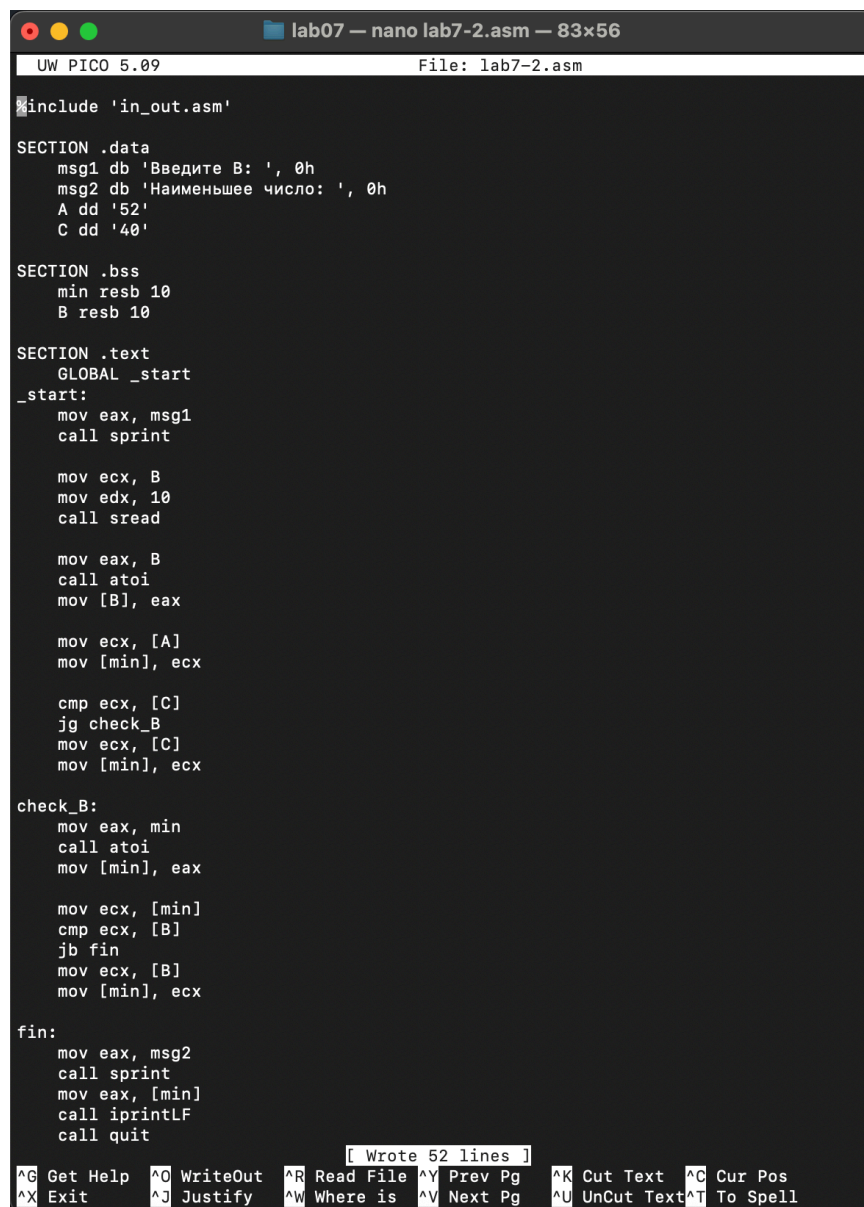
18 000000ED EB10FFFFFF call sprint
19
20 000000F2 B0[0A000000] mov ecx, B
21 000000F7 BA0A000000 mov edx, 10
22 000000FC E842FFFFFF call sread
23
24 00000101 B8[0A000000] mov eax, B
25 00000106 E891FFFFFF call atoi
26 0000010B A3[0A000000] mov [B], eax
27
28 00000110 8B0D[36000000] mov ecx, [A]
29 00000116 890D[00000000] mov [max], ecx
30
31 0000011C 3B0D[3A000000] cmp ecx, [C]
32 00000122 7F0C jg check_B
33 00000124 8B0D[3A000000] mov ecx, [C]
34 0000012A 890D[00000000] mov [max], ecx
35
36 check_B:
37 mov eax,
38 00000130 E867FFFFFF call atoi
39 00000135 A3[00000000] mov [max], eax
40
41 0000013A 8B0D[00000000] mov ecx, [max]
42 00000140 3B0D[0A000000] cmp ecx, [B]
43 00000146 7F0C jg fin
44 00000148 8B0D[0A000000] mov ecx, [B]
45 0000014E 890D[00000000] mov [max], ecx
46
47 fin:
48 00000154 B8[14000000] mov eax, msg2
49 00000159 E8B1FFFFFF call sprint
50 0000015E A1[00000000] mov eax, [max]
51 00000163 E81EFFFFFF call iprintLF
52 00000168 E86EFFFFFF call quit
53
54
55
error: invalid combination of opcode and operands

```

Рис. 4.12: Просмотр ошибки в файле листинга

4.3 Задания для самостоятельной работы

Искренне не понимаю, какой вариант я должен был получить во время 7 лабораторной работы, поэтому буду использовать свой 8 вариант - из предыдущей лабораторной работы. Возвращаю операнд к функции в программе и изменяю ее так, чтобы она выводила переменную с наименьшим значением (рис. 4.13).



```
UW PICO 5.09                                File: lab7-2.asm
#include 'in_out.asm'

SECTION .data
    msg1 db 'Введите B: ', 0h
    msg2 db 'Наименьшее число: ', 0h
    A dd '52'
    C dd '40'

SECTION .bss
    min resb 10
    B resb 10

SECTION .text
    GLOBAL _start
_start:
    mov eax, msg1
    call sprint

    mov ecx, B
    mov edx, 10
    call sread

    mov eax, B
    call atoi
    mov [B], eax

    mov ecx, [A]
    mov [min], ecx

    cmp ecx, [C]
    jg check_B
    mov ecx, [C]
    mov [min], ecx

check_B:
    mov eax, min
    call atoi
    mov [min], eax

    mov ecx, [min]
    cmp ecx, [B]
    jb fin
    mov ecx, [B]
    mov [min], ecx

fin:
    mov eax, msg2
    call sprint
    mov eax, [min]
    call iprintLF
    call quit

[ Wrote 52 lines ]
^G Get Help  ^O WriteOut  ^R Read File  ^Y Prev Pg   ^K Cut Text   ^C Cur Pos
^X Exit      ^J Justify   ^W Where is   ^V Next Pg   ^U UnCut Text ^T To Spell
```

Рис. 4.13: Первая программа самостоятельной работы

Код первой программы:

```

#include 'in_out.asm'

SECTION .data
    msg1 db 'Введите B: ', 0h
    msg2 db 'Наименьшее число: ', 0h
    A dd '52'
    C dd '40'

SECTION .bss
    min resb 10
    B resb 10

SECTION .text
    GLOBAL _start
_start:
    mov eax, msg1
    call sprint

    mov ecx, B
    mov edx, 10
    call sread

    mov eax, B
    call atoi
    mov [B], eax

    mov ecx, [A]
    mov [min], ecx

    cmp ecx, [C]
    jg check_B
    mov ecx, [C]
    mov [min], ecx

check_B:
    mov eax, min
    call atoi
    mov [min], eax

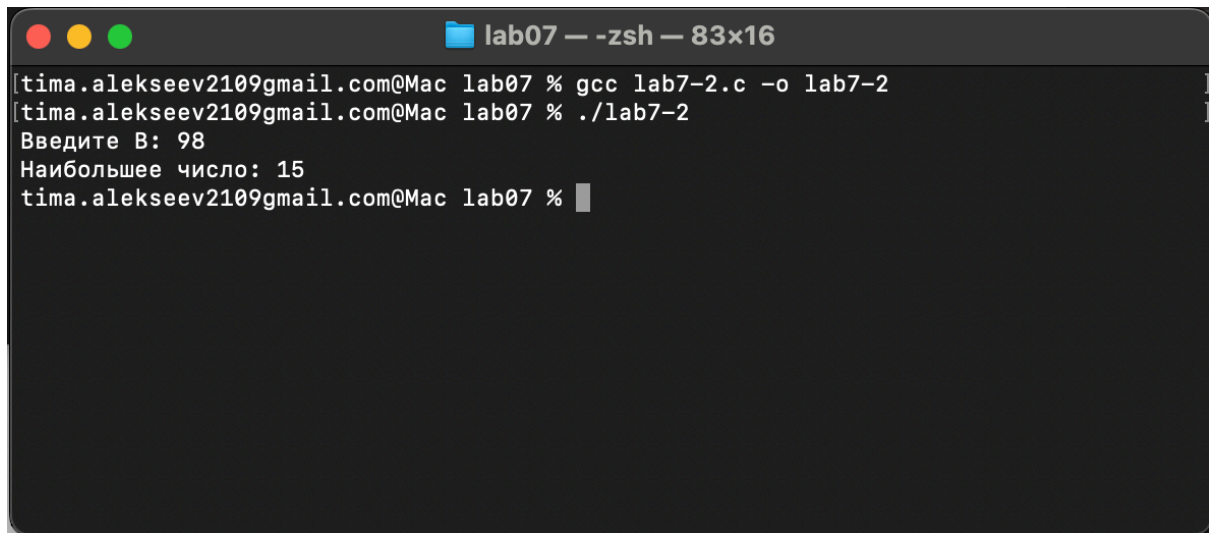
    mov ecx, [min]
    cmp ecx, [B]
    jb fin
    mov ecx, [B]
    mov [min], ecx

fin:
    mov eax, msg2
    call sprint
    mov eax, [min]

```

```
call iprintLF  
call quit
```

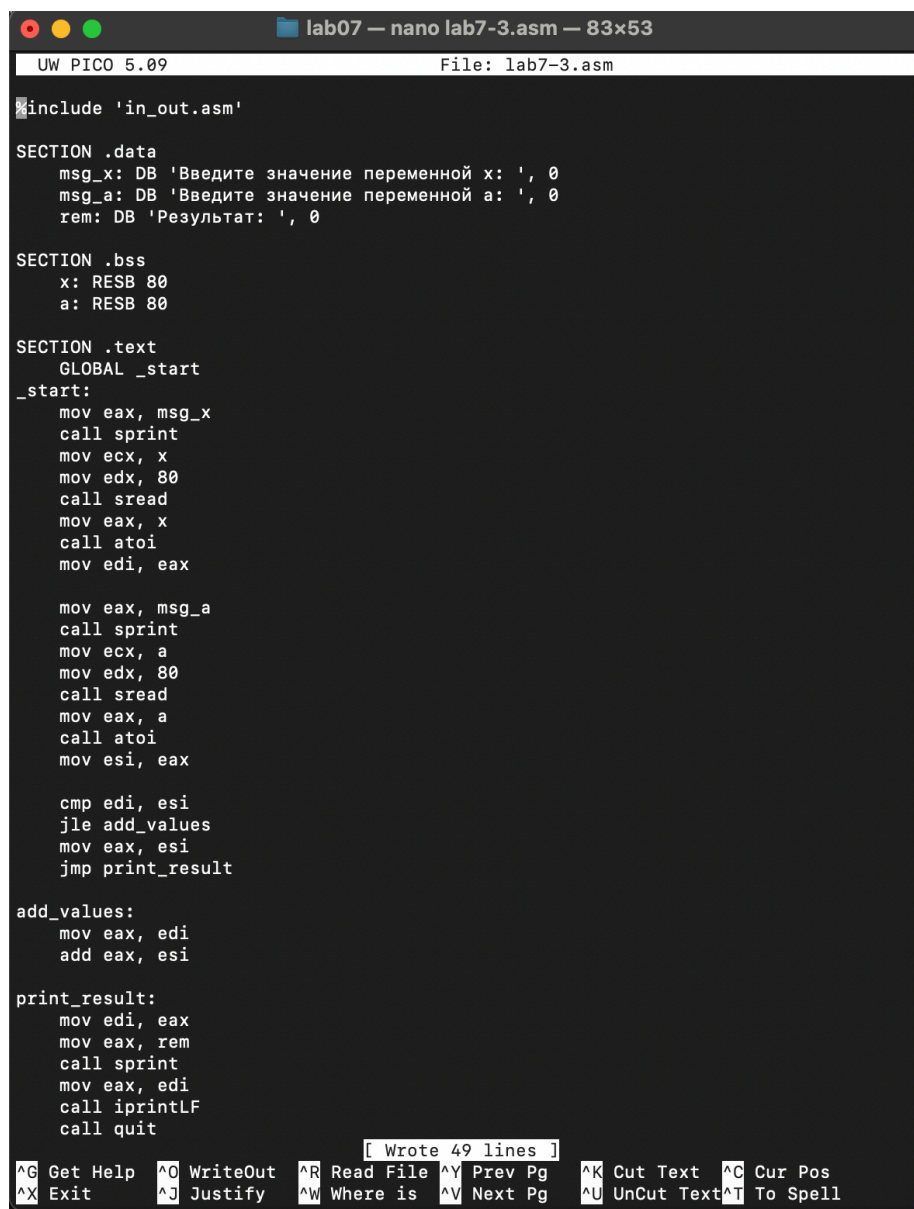
Проверяю корректность написания первой программы (рис. 4.14).

A screenshot of a macOS terminal window titled "lab07 — -zsh — 83x16". The terminal shows the following commands and output:

```
[tima.alekseev2109gmail.com@Mac lab07 % gcc lab7-2.c -o lab7-2  
[tima.alekseev2109gmail.com@Mac lab07 % ./lab7-2  
Введите В: 98  
Наибольшее число: 15  
tima.alekseev2109gmail.com@Mac lab07 %
```

Рис. 4.14: Проверка работы первой программы

Пишу программу, которая будет вычислять значение заданной функции согласно моему варианту для введенных с клавиатуры переменных а и х (рис. 4.15).



```
lab07 — nano lab7-3.asm — 83x53
UW PICO 5.09 File: lab7-3.asm

#include 'in_out.asm'

SECTION .data
msg_x: DB 'Введите значение переменной x: ', 0
msg_a: DB 'Введите значение переменной a: ', 0
rem: DB 'Результат: ', 0

SECTION .bss
x: RESB 80
a: RESB 80

SECTION .text
GLOBAL _start
_start:
    mov eax, msg_x
    call sprint
    mov ecx, x
    mov edx, 80
    call sread
    mov eax, x
    call atoi
    mov edi, eax

    mov eax, msg_a
    call sprint
    mov ecx, a
    mov edx, 80
    call sread
    mov eax, a
    call atoi
    mov esi, eax

    cmp edi, esi
    jle add_values
    mov eax, esi
    jmp print_result

add_values:
    mov eax, edi
    add eax, esi

print_result:
    mov edi, eax
    mov eax, rem
    call sprint
    mov eax, edi
    call iprintLF
    call quit

[ Wrote 49 lines ]
^G Get Help ^O WriteOut ^R Read File ^Y Prev Pg ^K Cut Text ^C Cur Pos
^X Exit ^J Justify ^W Where is ^V Next Pg ^U UnCut Text ^T To Spell
```

Рис. 4.15: Вторая программа самостоятельной работы

Код второй программы:

```
%include 'in_out.asm'

SECTION .data
    msg_x: DB 'Введите значение переменной x: ', 0
    msg_a: DB 'Введите значение переменной a: ', 0
    rem: DB 'Результат: ', 0

SECTION .bss
    x: RESB 80
    a: RESB 80

SECTION .text
    GLOBAL _start
_start:
    mov eax, msg_x
```

```

call sprint
mov ecx, x
mov edx, 80
call sread
mov eax, x
call atoi
mov edi, eax

mov eax, msg_a
call sprint
mov ecx, a
mov edx, 80
call sread
mov eax, a
call atoi
mov esi, eax

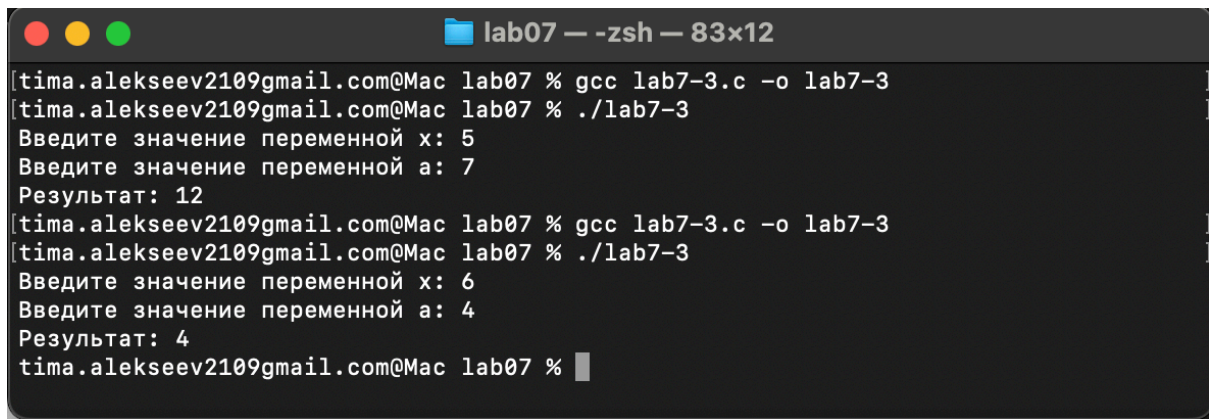
cmp edi, esi
jle add_values
mov eax, esi
jmp print_result

add_values:
mov eax, edi
add eax, esi

print_result:
mov edi, eax
mov eax, rem
call sprint
mov eax, edi
call iprintLF
call quit

```

Транслирую и компоную файл, запускаю и проверяю работу программы для различных значений а и х (рис. 4.16).



A terminal window titled "lab07 — -zsh — 83x12" with standard macOS window controls (red, yellow, green buttons). The terminal shows the compilation and execution of a C program named "lab7-3". The user enters the command "gcc lab7-3.c -o lab7-3" to compile the program. Then, they run "./lab7-3". The program prompts for two inputs: "Введите значение переменной x: 5" and "Введите значение переменной a: 7". It then outputs "Результат: 12". The user repeats the process with inputs 6 and 4, resulting in "Результат: 4". The terminal ends with the prompt "tima.alekseev2109gmail.com@Mac lab07 %".

```
[tima.alekseev2109gmail.com@Mac lab07 % gcc lab7-3.c -o lab7-3  
[tima.alekseev2109gmail.com@Mac lab07 % ./lab7-3  
Введите значение переменной x: 5  
Введите значение переменной a: 7  
Результат: 12  
[tima.alekseev2109gmail.com@Mac lab07 % gcc lab7-3.c -o lab7-3  
[tima.alekseev2109gmail.com@Mac lab07 % ./lab7-3  
Введите значение переменной x: 6  
Введите значение переменной a: 4  
Результат: 4  
tima.alekseev2109gmail.com@Mac lab07 % █
```

Рис. 4.16: Проверка работы второй программы

5. Выводы

При выполнении лабораторной работы я изучил команды условных и безусловных переходов, а также приобрел навыки написания программ с использованием переходов, познакомился с назначением и структурой файлов листинга.

6. Список литературы

1. Курс на ТУИС
2. Лабораторная работа №7
3. Программирование на языке ассемблера NASM Столяров А. В.